

Optimizadores

Los **optimizadores** son los algoritmos que controlan cómo se actualizan los pesos de una red neuronal durante el entrenamiento, basándose en los gradientes calculados por **backpropagation**. Elegir el optimizador adecuado puede tener un gran impacto en la velocidad de convergencia, la estabilidad del entrenamiento y el rendimiento final del modelo.

¿Qué hace un optimizador?

Un optimizador busca **minimizar la función de pérdida**, es decir, encontrar los valores de los pesos que hacen que el modelo cometa menos errores. Para hacerlo, usa información sobre:

- El valor actual de los pesos.
 - El gradiente de la pérdida respecto a esos pesos.
 - Información histórica (en algunos casos).
-

Tipos comunes de optimizadores

Aquí te presento los optimizadores más usados, con sus ventajas, desventajas y cuándo usar cada uno:

1. SGD (Stochastic Gradient Descent)

Descripción:

Actualiza los pesos usando el gradiente de un mini-batch (una pequeña muestra del dataset) en lugar del conjunto completo.

Ventajas:

- Simple y fácil de entender.
- Puede escapar mínimos locales debido al ruido del mini-batch.

Desventajas:

- Lento para converger.
- Oscila mucho alrededor del mínimo óptimo.

Parámetros típicos:

- `learning_rate`: tasa de aprendizaje fija.

17 Cuándo usarlo:

- En problemas simples o como base para otros optimizadores.
 - Si tienes recursos limitados o necesitas total control sobre la actualización de pesos.
-

2. SGD con Momentum

Descripción:

Extiende SGD añadiendo "inercia", es decir, mantiene una velocidad acumulada del movimiento en cierta dirección.

Ventajas:

- Acelera la convergencia en direcciones relevantes.
- Reduce oscilaciones en valles curvados.

Desventajas:

- Puede sobrepasar el mínimo si no se ajusta bien el momentum.

Parámetros típicos:

- `learning_rate`
- `momentum` (típicamente 0.9)

17 Cuándo usarlo:

- En redes profundas cuando buscas mejor convergencia que con SGD básico.
 - Especialmente útil en problemas con superficies de pérdida con valles o formas irregulares.
-

3. RMSProp (Root Mean Square Propagation)

Descripción:

Ajusta dinámicamente la tasa de aprendizaje para cada peso según la magnitud reciente de sus gradientes.

Ventajas:

- Bueno para problemas no convexos y con gradientes variables.
- Controla mejor las explosiones de gradiente.

Desventajas:

- Menos popular hoy en día por Adam.
- Requiere ajuste fino de parámetros.

Parámetros típicos:

- `learning_rate`
- `rho` (factor de decaimiento para promedio móvil)

17 Cuándo usarlo:

- En RNNs o modelos recurrentes.
 - Cuando trabajas con datos secuenciales o funciones de pérdida ruidosas.
-

4. Adam (Adaptive Moment Estimation)

Descripción:

Combina ideas de RMSProp y Momentum: usa estimaciones adaptativas de momentos de primer y segundo orden (media y varianza de los gradientes).

Ventajas:

- Muy estable y rápido.
- Funciona bien en muchos tipos de problemas.
- No requiere ajuste fino.

Desventajas:

- Puede ser menos robusto en ciertos problemas de optimización complejos.
- Algunas investigaciones sugieren que generaliza peor que SGD con momentum en ciertos casos.

Parámetros típicos:

- `learning_rate` (por defecto: 0.001)
- `beta_1` , `beta_2` (momentos, por defecto: 0.9 y 0.999)
- `epsilon` (evita división por cero, por defecto: 1e-7)

17 Cuándo usarlo:

- Es el optimizador por defecto en muchos frameworks (como Keras o PyTorch).
- Ideal para empezar a entrenar cualquier modelo moderno.
- Bueno para problemas con datos dispersos (ej: NLP).

5. AdamW (Adam with Weight Decay)

Descripción:

Una variante de Adam que aplica correctamente la regularización L2 (weight decay), algo que Adam original no hacía del todo bien.

Ventajas:

- Mejora la generalización del modelo.
- Más eficaz en entrenamientos largos con regularización.

Desventajas:

- Similar a Adam, pero con más configuración.

17 Cuándo usarlo:

- En entrenamientos largos donde la regularización importa.
- En arquitecturas grandes como transformers o CNNs profundas.

6. LAMB (Layer-wise Adaptive Moments optimizer for Batch training)

Descripción:

Versión escalable de Adam diseñada para entrenamiento con **batch sizes muy grandes**.

Ventajas:

- Escalable a GPUs/TPUs y batch sizes gigantes.
- Evita actualizaciones excesivas en capas con normas grandes.

Desventajas:

- Más complejo.
- Solo útil en escenarios de alto rendimiento.

17 Cuándo usarlo:

- Entrenamiento distribuido de modelos muy grandes (ej: BERT, GPT).
 - En entornos con múltiples GPUs o TPUs.
-

7. Nadam (Nesterov-accelerated Adaptive Moment Estimation)

Descripción:

Combinación de **Adam** y **NAG (Nesterov Accelerated Gradient)**, mejora el momentum.

Ventajas:

- Más sensible a cambios futuros en la dirección del gradiente.
- Buena convergencia.

Desventajas:

- Similar a Adam, sin mejoras significativas en muchos casos.

17 Cuándo usarlo:

- En problemas donde el momentum tradicional no es suficiente.
 - Modelos pequeños o medianos con estructura compleja.
-

8. Adagrad

Descripción:

Asigna tasas de aprendizaje más pequeñas a características frecuentes y más altas a raras.

Ventajas:

- Excelente para datos dispersos (ej: NLP, recomendadores).
- No necesita ajustar manualmente la tasa de aprendizaje.

Desventajas:

- La tasa de aprendizaje disminuye con el tiempo y eventualmente se vuelve demasiado pequeña.

17 Cuándo usarlo:

- Problemas con datos muy dispersos.
- Sistemas de recomendación, embeddings, etc.

9. Adadelta

Descripción:

Mejora de Adagrad que evita la caída progresiva de la tasa de aprendizaje mediante una ventana móvil de gradientes pasados.

Ventajas:

- No requiere tasa de aprendizaje explícita.
- Más estable que Adagrad.

Desventajas:

- Menos usado hoy en día por Adam.

17 Cuándo usarlo:

- Como alternativa a Adagrad, aunque rara vez es el primero a elegir.

Cuál usar y cuándo: guía rápida

Optimizador	Caso ideal	Notas
SGD	Principiantes, modelos sencillos	Lento pero simple
SGD + Momentum	Redes profundas, convergencia mejorada	Muy común en visiones clásicas
Adam	General-purpose, modelos modernos	Por defecto en la mayoría de frameworks
AdamW	Regularización importante	Mejor generalización
RMSProp	RNNs o datos ruidosos	Más usado en el pasado
LAMB	Modelos enormes, batch grande	Útil en entrenamiento masivo
Adagrad	Datos dispersos (NLP)	Útil en embeddings
Adadelta	Sin tasa de aprendizaje	Menos usado ahora

Recomendaciones prácticas

1. **Para empezar:** Usa **Adam** → es robusto, rápido y funciona bien en la mayoría de casos.
2. **Si buscas mejor generalización:** Prueba **AdamW** con weight decay.
3. **En redes muy profundas:** **SGD con momentum** puede dar mejores resultados.
4. **En modelos recurrentes (RNN/LSTM):** **RMSProp** o **Adam**.
5. **En entrenamiento distribuido:** **LAMB** para batches muy grandes.
6. **En datos dispersos (ej: NLP):** **Adagrad** o **Adam**.